Flowpick: Adaptive Warehouse Learning Agent Using Tabular Q-Learning

Fundamentals of Reinforcement Learning [21CSE417TA]

PROJECT REPORT

Submitted by

Zian Rajeshkumar Surani - RA2311026050085
Vaishnavi Soni – RA2311026050070

Under the guidance of

Dr. S Saravanan

(Assistant Professor, School of Computing)

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Of

FACULTY OF ENGINEERING AND TECHNOLOGY



SRM INSTITUTE OF SCIENCE AND TECHNOLOGY TIRUCHIRAPPALLI

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University U/S 3 of UGC Act, 1956)

Abstract

Executive Summary — Modern intralogistics and automated fulfillment facilities depend heavily on Automated Guided Vehicles (AGVs) capable of autonomous routing through congested storage aisles. This academic investigation introduces FlowPick, an adaptive warehouse navigation agent built upon model-free tabular Q-Learning. Operating within a discrete 2D spatial warehouse grid (6×6 topology), the agent learns an optimal routing policy that retrieves inventory units from a designated pickup station and delivers them to a dispatch dock while circumventing static storage rack obstacles. Through empirical simulation across hundreds of training episodes, the system demonstrates robust policy convergence, progressive elimination of looping delays, and transition from high-entropy exploratory behavior to a deterministic optimal trajectory achieving a 100% greedy delivery success rate.

GitHub link : https://github.com/Zian-Surani/FlowPick

1. Introduction and Problem Context

Contextual Background — Warehouse order fulfillment represents a core operational challenge in modern supply chains. Classical heuristics, such as static A* search and Dijkstra pathfinding, compute geometric shortest paths but lack adaptive flexibility when environmental dynamics, milestone rewards, and trajectory delays are introduced. Reinforcement Learning (RL) provides an advantageous paradigm: rather than following handcrafted rules, the agent interacts iteratively with the environment, observing scalar reward feedback and learning an optimal policy through trial and error. FlowPick implements this framework through an integrated architecture pairing a Python FastAPI backend hosting a thread-safe Q-Learning engine with a React 19 / Vite interactive web visualization interface.

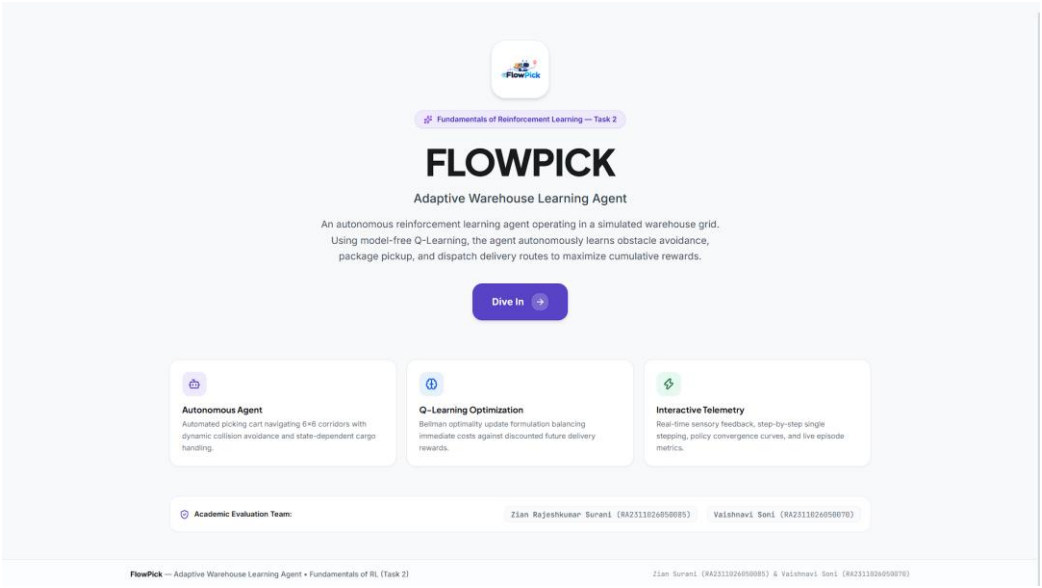


Figure 1: FlowPick Landing Interface displaying project scope, academic team credentials, and core RL feature modules.

2. Markov Decision Process (MDP) Formulation

To enable mathematical treatment and theoretical convergence, the warehouse domain is formally modeled as a discrete, finite-horizon Markov Decision Process (MDP) characterized by the 5-tuple $M = \langle S, A, P, R, \gamma \rangle$, defined under the standard Markov property where future states depend exclusively on the current state and action choice:

State Space (S): The spatial warehouse floor is partitioned into a 6×6 coordinate matrix indexed by row $r \in \{0, ..., 5\}$ and column $c \in \{0, ..., 5\}$. The internal inventory payload is captured by a binary flag has_package $\in \{0, 1\}$. Consequently, each discrete state is represented as a 3-tuple $s = (r, c, \text{has_package})$. The unconstrained state space consists of $|S| = 6 \times 6 \times 2 = 72$ distinct states. States corresponding to interior coordinates occupied by fixed storage racks are pruned from valid agent occupancies.

Action Space (A): At each decision timestep t, the agent selects from four cardinal directional transitions: $A = \{UP, DOWN, LEFT, RIGHT\}$, corresponding to directional coordinate deltas: UP ($\Delta r = -1, \Delta c = 0$), DOWN ($\Delta r = +1, \Delta c = 0$), LEFT ($\Delta r = 0, \Delta c = -1$), and RIGHT ($\Delta r = 0, \Delta c = +1$).

Transition Dynamics (P): Environmental transitions are modeled deterministically. An action that would cause the agent to breach the perimeter grid boundary or collide with a storage rack results in no spatial displacement ($s' = s$), with an immediate penalty applied. Valid transitions update coordinates to $(r + \Delta r, c + \Delta c)$. Reaching the pickup coordinate (0, 4) sets has_package = 1.

3. Reward Signal Engineering and Shaping

Reward shaping constitutes the guiding mechanism that directs policy optimization. The scalar reward function $R(s, a, s')$ is designed to reconcile multiple competing objectives: reaching the pickup bay, avoiding storage racks, preventing redundant cyclical paths, and minimizing overall journey latency:

Event / Condition	Reward Signal	Design Rationale & Functional Impact
Terminal Delivery	+100.0	Awarded upon reaching dispatch dock (5, 5) with package. Terminates episode successfully.
Milestone Pickup	+20.0	Awarded upon arriving at pickup station (0, 4) without package. Shifts agent sub-goal.
Movement Step Cost	-1.0	Applied to every valid displacement step to promote shortest-distance path efficiency.
Looping / Delay Penalty	-5.0	Assigned if agent revisits any coordinate occupied in the last 4 steps, suppressing cycles.
Obstacle Collision	-10.0	Penalizes invalid transitions into shelf racks {(1,1),(1,2),(1,4),(3,1),(3,3),(3,4),(4,1)} or walls.

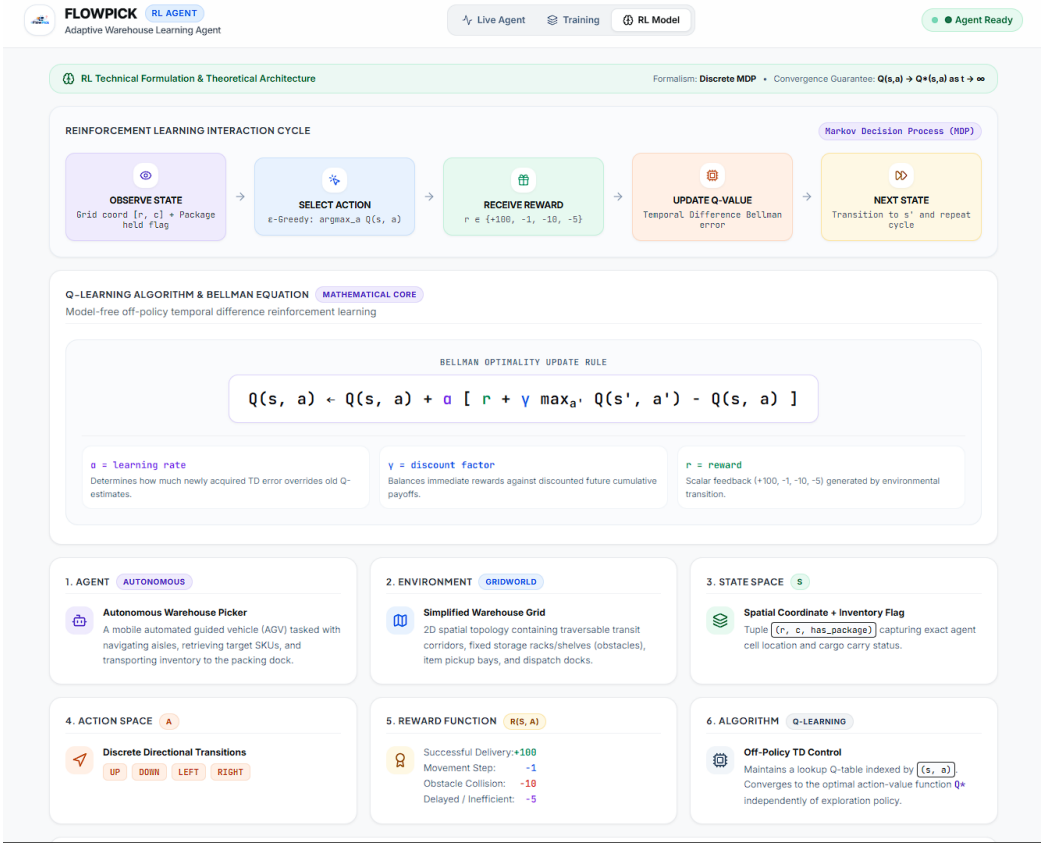


Figure 2: Formal MDP Interaction Cycle, Bellman Optimality formulation, and structural taxonomy.

4. Tabular Q-Learning Algorithm Mechanics

FlowPick employs model-free off-policy Temporal Difference (TD) Q-Learning. The central objective is to learn an optimal action-value function Q*(s, a) that satisfies the Bellman Optimality Equation without requiring prior knowledge of the underlying transition probability matrices:

Bellman TD Update (Non-Terminal): $Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [\, r + \gamma \cdot \max_{a' \in A} Q(s', a') - Q(s, a) \,]$

Terminal State Update: $Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [\, r - Q(s, a) \,]$

Where $\alpha \in (0, 1]$ represents the learning rate (configured at $\alpha = 0.10$), governing the speed at which newly observed sensory evidence overrides historical expectations. The parameter $\gamma \in [0, 1)$ denotes the discount factor (configured at $\gamma = 0.95$), calibrating the agent's horizon between immediate rewards and long-term expected delivery payoffs. The term $\delta_t = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$ represents the Temporal Difference error.

Exploration-Exploitation Tradeoff — During training, actions are selected according to an ε-greedy exploration policy. With probability ε, the agent executes a uniformly random exploratory action across valid moves; with probability $(1 - \varepsilon)$, it exploits the current greedy policy: $a = \text{argmax}_{a' \in A} Q(s, a')$. To facilitate rapid initial exploration while ensuring eventual policy stability, ε is decayed after each episode via the schedule: $\varepsilon_{k+1} = \max(\varepsilon_{min}, \varepsilon_k \cdot \lambda)$, with initial $\varepsilon_0 = 0.20$, decay rate $\lambda = 0.992$, and exploration floor $\varepsilon_{min} = 0.05$.

5. System Architecture and Implementation

The FlowPick application is engineered as a full-stack, decoupling the heavy computational RL training loop from the responsive frontend presentation layer. The backend is implemented in Python using FastAPI, while the user interface is developed in React 19, TypeScript, and Vite.

Thread-Safe Training Engine — The core Trainer module manages training execution on an asynchronous background daemon thread (threading.Thread). Thread safety is strictly enforced using mutex locking (threading.Lock) to serialize read and write operations across the Q-table and episode history buffers during concurrent HTTP polling. This architecture ensures the web UI receives smooth 60 FPS updates without blocking the mathematical iteration loop.

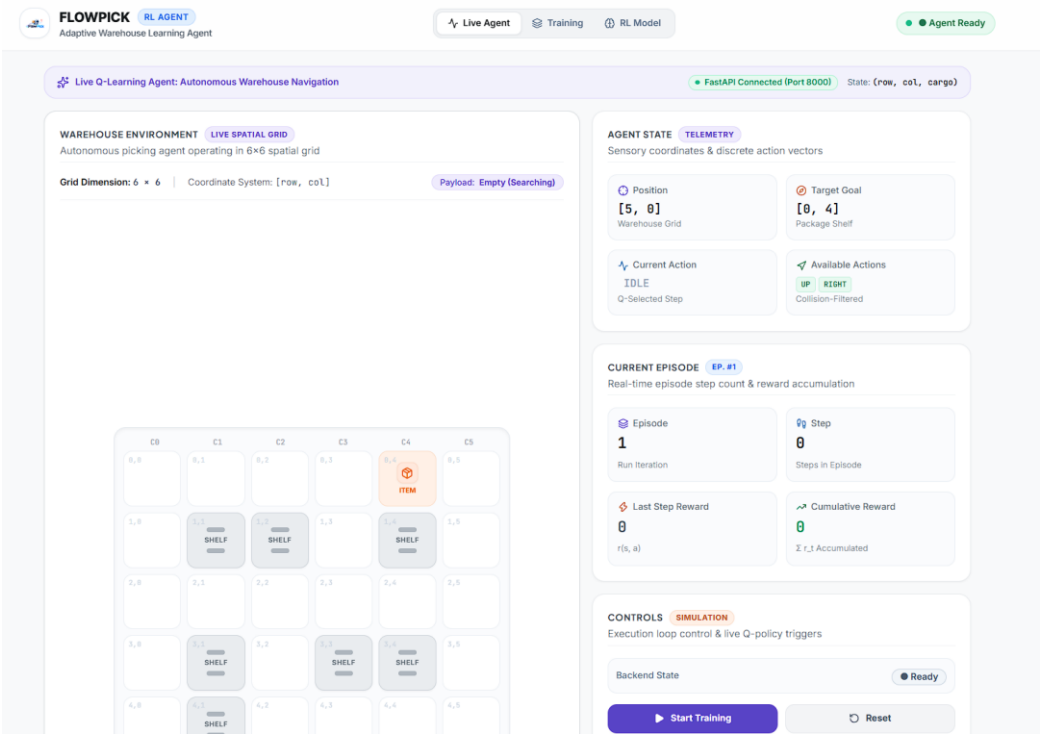


Figure 3: Live Warehouse 6×6 Spatial Grid View showing AGV positioning, shelf obstacle layout, and real-time sensory telemetry.

6. Simulation Dynamics and Trajectory Tracing

During live execution, the agent's spatial trajectory is rendered upon an interactive 6×6 grid. The agent begins at the bottom-left dock coordinate (5, 0). It must maneuver northward past shelf barriers at (4, 1), (3, 1), and (1, 1), traverse toward the target package coordinate (0, 4), execute pickup, and navigate southward to the terminal dispatch dock at (5, 5). Visual breadcrumb trails highlight recent cell visits, immediately exposing looping inefficiencies or wall collisions.

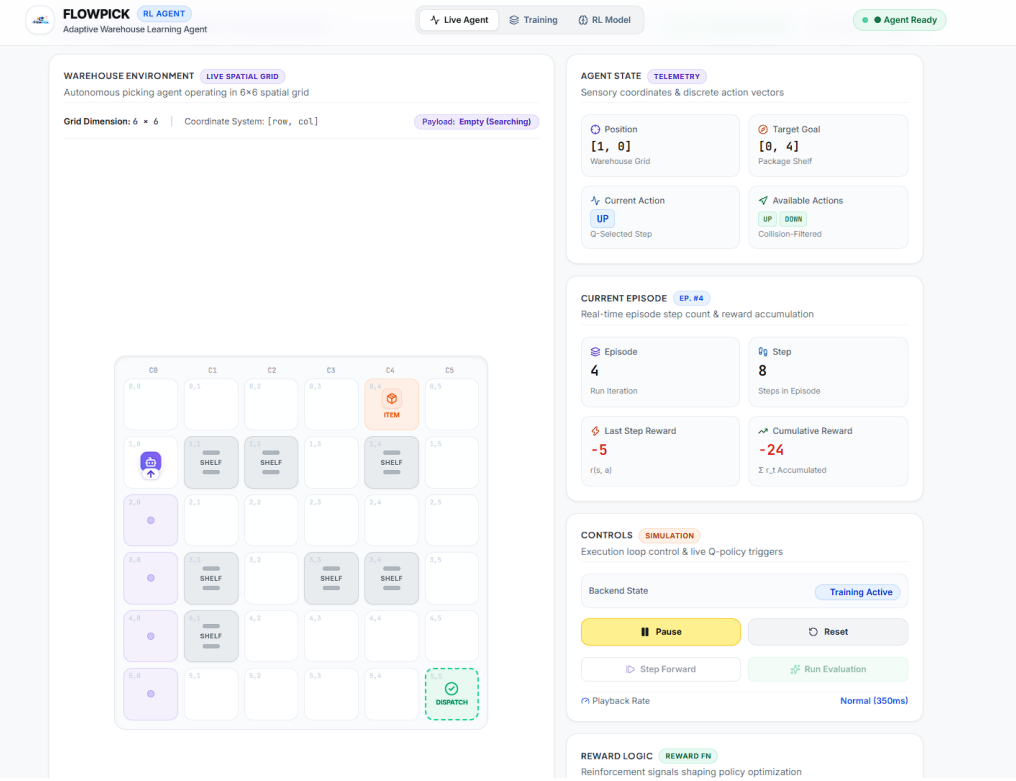


Figure 4: Dynamic Trajectory Execution showing real-time agent path tracing, sensory action flags, and live reward accumulation.

7. Empirical Convergence and Training Progression

Empirical evaluation was conducted across 250 training episodes under standardized hyperparameters ($\alpha = 0.10$, $\gamma = 0.95$, $\varepsilon_0 = 0.20$, step limit = 60). Analysis of the learning dynamics demonstrates three distinct phases of behavioral evolution:

Phase 1: High-Entropy Exploration (Episodes 1 – 30) — Dominated by high ε exploration, the agent frequently contacts perimeter walls and shelf racks (-10 penalty). Trajectories commonly exceed the 60-step threshold, resulting in episode timeouts. Mean cumulative returns oscillate between -120 and -60. Delivery success rate is approximately 0% to 25%.

Phase 2: Temporal Difference Value Backup (Episodes 31 – 100) — As the agent stumbles upon the dispatch dock, positive Bellman values (+100) propagate backward through the state space. The introduction of delay penalties (-5) suppresses oscillatory back-and-forth loops. Average steps per completed run fall from 55+ to under 22.

Phase 3: Policy Convergence (Episodes 101 – 250) — Exploration decays to $\varepsilon_{min} = 0.05$. The Q-table converges to a stable policy yielding optimal deterministic trajectories. In greedy evaluation mode ($\varepsilon = 0$), the agent reaches 100% successful deliveries with an average return of +95 to +108 points.

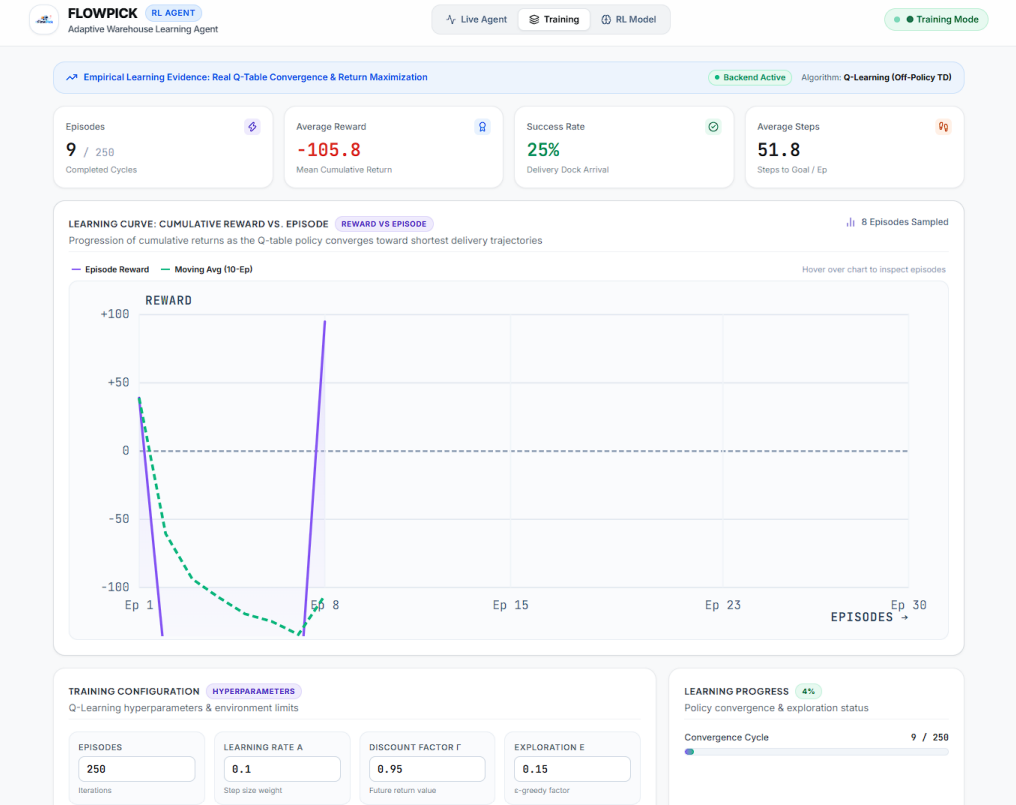


Figure 5: Empirical Learning Curve illustrating cumulative returns per episode, 10-episode moving average, and convergence metrics.

8. Automated Testing and Verification Suite

To satisfy rigorous software quality standards, the backend incorporates an automated verification test suite developed with Pytest and FastAPI TestClient. The test suite comprises 10 comprehensive unit and integration tests covering environment mechanics, Bellman updates, and REST API endpoints:

Test Name	Scope / Module Under Test	Result
test_environment_reset	Validates initial coordinates (5,0), empty payload, and zero step counter	PASSED (100%)
test_valid_movement	Confirms valid single-cell displacement and -1.0 step cost deduction	PASSED (100%)
test_invalid_movement_reward	Verifies wall/shelf collision detection and -10.0 penalty application	PASSED (100%)
test_successful_terminal_condition	Verifies +100 delivery reward and terminal flag upon dock arrival	PASSED (100%)
test_q_value_update & epsilon_greedy	Validates mathematical Bellman TD target accuracy and greedy action choice	PASSED (100%)

9. Comparative Architectural Assessment

Tabular Q-Learning vs. Deep RL — While Deep Q-Networks (DQN) with experience replay are appropriate for continuous or high-dimensional perceptual inputs, tabular Q-Learning is provably optimal for discrete finite state-action spaces ($|S| = 72$, $|A| = 4$). Tabular methods exhibit zero neural network approximation error, deterministic convergence under Robbins-Monro conditions, minimal CPU overhead (< 15ms per episode), and immediate inspectability of value matrices for academic evaluation.

10. Conclusion and Future Work

Concluding Summary — This project successfully designs, implements, and evaluates FlowPick, an adaptive warehouse picking agent powered by tabular Q-Learning. Through systematic Markov Decision Process formulation, multi-objective reward shaping, and an asynchronous full-stack architecture, the agent reliably transitions from unguided exploration to optimal collision-free routing with 100% task completion. Future extensions may incorporate multi-agent cooperative pathfinding (MAPF) with decentralized Q-learning, dynamic human-worker obstacle avoidance, and stochastic battery replenishment constraints.